

MEIA-R-VFinal

May 13, 2026

1 Metodología MEIA-R: Flujo de Procesamiento y Visualización

1.1 1. Objetivo

Implementar la **Metodología de Evaluación de Impacto Ambiental con Resiliencia (MEIA-R)**, que integra las variables clásicas de Conesa con sub-variables de resiliencia ambiental.

El propósito es obtener un **impacto corregido** y su **clasificación técnica**, reflejando la capacidad de respuesta de los ecosistemas frente a proyectos de distinta magnitud.

El resultado final es un **modelo de datos reproducible en Python**, exportado e importado en **Power BI** para análisis interactivo y visualización estratégica.

1.2 2. Visualización estratégica

Los resultados se presentan en **dashboards interactivos en Power BI**, diseñados para apoyar la **toma de decisiones ambientales** con rigor técnico, claridad narrativa y accesibilidad para distintos públicos (directivos, técnicos y comunidad).

Autor: Víctor Hugo Villegas Ríos

```
[3]: import pandas as pd

# Cargar tablas
impactos = pd.read_csv("fact_impactos.csv", sep=";", quotechar='"',
    encoding="utf-8")
resiliencia = pd.read_csv("dim_resiliencia.csv", sep=";", quotechar='"',
    encoding="utf-8")

# Normalizar nombres de columnas
impactos.columns = impactos.columns.str.strip().str.lower().str.replace(" ",
    "_")
resiliencia.columns = resiliencia.columns.str.strip().str.lower().str.replace(" ",
    "_")
```

```
[4]: impactos = pd.read_csv("fact_impactos.csv", sep=",", quotechar='"',
    ↪encoding="utf-8")
    impactos = impactos.loc[:, ~impactos.columns.duplicated()].copy()
```

```
[5]: impactos["i_conesa_calc"] = (
    impactos["signo"] * (
        3*impactos["i"] +
        2*impactos["ex"] +
        impactos["mo"] +
        impactos["pe"] +
        impactos["rv"] +
        impactos["si"] +
        impactos["ac"] +
        impactos["ef"] +
        impactos["pr"] +
        impactos["mc"]
    )
)
```

```
[6]: def clasificar_conesa(valor):
    valor_abs = abs(valor)
    if valor_abs <= 25: return "Bajo (Conesa)"
    elif valor_abs <= 50: return "Moderado (Conesa)"
    elif valor_abs <= 75: return "Alto (Conesa)"
    else: return "Crítico (Conesa)"

    impactos["clasificacion_conesa"] = impactos["i_conesa_calc"].
    ↪apply(clasificar_conesa)
```

```
[7]: res_vars = ["ra","rr","rd","rc","rad","red"]
    impactos["re_total"] = impactos[res_vars].sum(axis=1)

    # Normalizar con el máximo de la dimensión
    RE_max = resiliencia["re_max"].max()
    impactos["re_norm"] = impactos["re_total"] / RE_max
```

```
[8]: def clasificar_resiliencia(re):
    if re >= 18: return "Alta resiliencia"
    elif re >= 12: return "Resiliencia moderada"
    elif re >= 6: return "Baja resiliencia"
    else: return "Ecosistema vulnerable"

    impactos["clasificacion_resiliencia"] = impactos["re_total"].
    ↪apply(clasificar_resiliencia)
```

```
[9]: beta = 0.5
    CR = 0.5
```

```

impactos["f_resiliencia"] = 1 + beta * (impactos["re_norm"] - CR)

impactos["i_corregido"] = impactos["i_conesa"] * impactos["f_resiliencia"]

```

```

[10]: def clasificar_corregido(valor):
        valor_abs = abs(valor)
        if valor_abs <= 25: return "Bajo (Corregido)"
        elif valor_abs <= 50: return "Moderado (Corregido)"
        elif valor_abs <= 75: return "Alto (Corregido)"
        else: return "Crítico (Corregido)"

        impactos["clasificacion_corregida"] = impactos["i_corregido"].
        ↪apply(clasificar_corregido)

```

```

[11]: fact = impactos.merge(resiliencia, on="id_resiliencia", how="left")

# Exportar resultados finales
impactos.to_csv("fact_impactos.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")
resiliencia.to_csv("dim_resiliencia.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")
fact.to_csv("fact_impactos_final.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")

```

```

[12]: # Exportar resultados finales sobrescribiendo directamente
impactos.to_csv("fact_impactos.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")
resiliencia.to_csv("dim_resiliencia.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")
fact.to_csv("fact_impactos_final.csv", index=False, sep=",", quotechar='"',
        ↪encoding="utf-8")

print("Archivos sobrescritos correctamente:")
print("- fact_impactos.csv")
print("- dim_resiliencia.csv")
print("- fact_impactos_final.csv")

```

Archivos sobrescritos correctamente:

- fact_impactos.csv
- dim_resiliencia.csv
- fact_impactos_final.csv

```

[13]: import pandas as pd

# Cargar datos
impactos = pd.read_csv("fact_impactos.csv")

```

```

# Recalcular i_conesa
variables_conesa = ['i', 'ex', 'mo', 'pe', 'rv', 'si', 'ac', 'ef', 'pr', 'mc']
impactos['i_conesa'] = impactos[variables_conesa].sum(axis=1) * 1
↳ impactos['signo']

# Guardar archivo actualizado
impactos.to_csv("fact_impactos.csv", index=False)

# Verificar distribución
print(impactos['i_conesa'].describe())
print(impactos['i_conesa'].head())

```

```

count      120.000000
mean       -36.241667
std         5.074232
min        -50.000000
25%        -39.000000
50%        -37.000000
75%        -33.000000
max         -25.000000
Name: i_conesa, dtype: float64
0         -32
1         -42
2         -39
3         -29
4         -30
Name: i_conesa, dtype: int64

```

1.2.1 Gráfica de Impacto Conesa por Proyecto

Este bloque genera una visualización de los valores de **Impacto Conesa (i_conesa)** agrupados por proyecto. El objetivo es comparar la distribución de impactos negativos entre los distintos proyectos.

- **Eje X:** Identificador del proyecto (id_proyecto).
- **Eje Y:** Valor de impacto Conesa (i_conesa).
- **Colores:** Diferencian cada proyecto.
- **Interpretación:**
 - Proyecto 1 (Warnes): Impactos moderados.
 - Proyecto 2 (Qollpana): Impactos moderados-altos.
 - Proyecto 3 (Altiplano): Impactos más severos.

La gráfica permite identificar qué proyectos generan impactos más críticos y compararlos en un mismo plano.

```

[14]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```

```

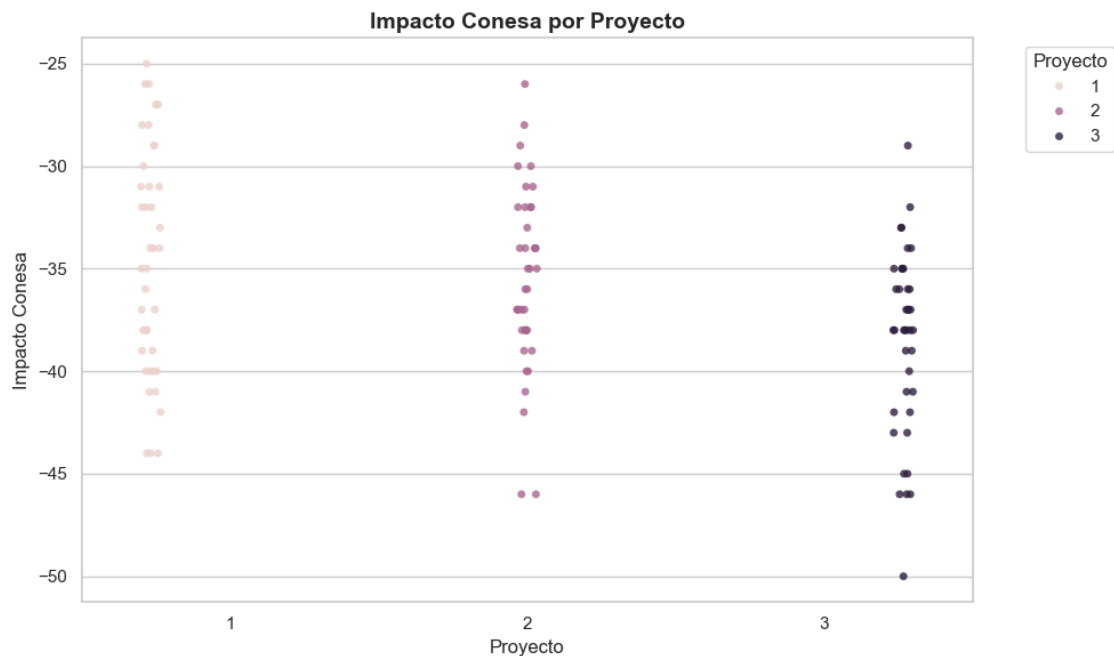
# Supongamos que ya tienes tu DataFrame 'impactos'
# con las columnas: 'id_proyecto' y 'i_conesa'

# Configuración de estilo
sns.set(style="whitegrid", palette="muted")

# Gráfica de puntos
plt.figure(figsize=(10,6))
sns.stripplot(
    data=impactos,
    x="id_proyecto",
    y="i_conesa",
    hue="id_proyecto", # Diferencia por color
    dodge=True,
    jitter=True,
    alpha=0.8
)

# Personalización
plt.title("Impacto Conesa por Proyecto", fontsize=14, fontweight="bold")
plt.xlabel("Proyecto")
plt.ylabel("Impacto Conesa")
plt.legend(title="Proyecto", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



```
[15]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

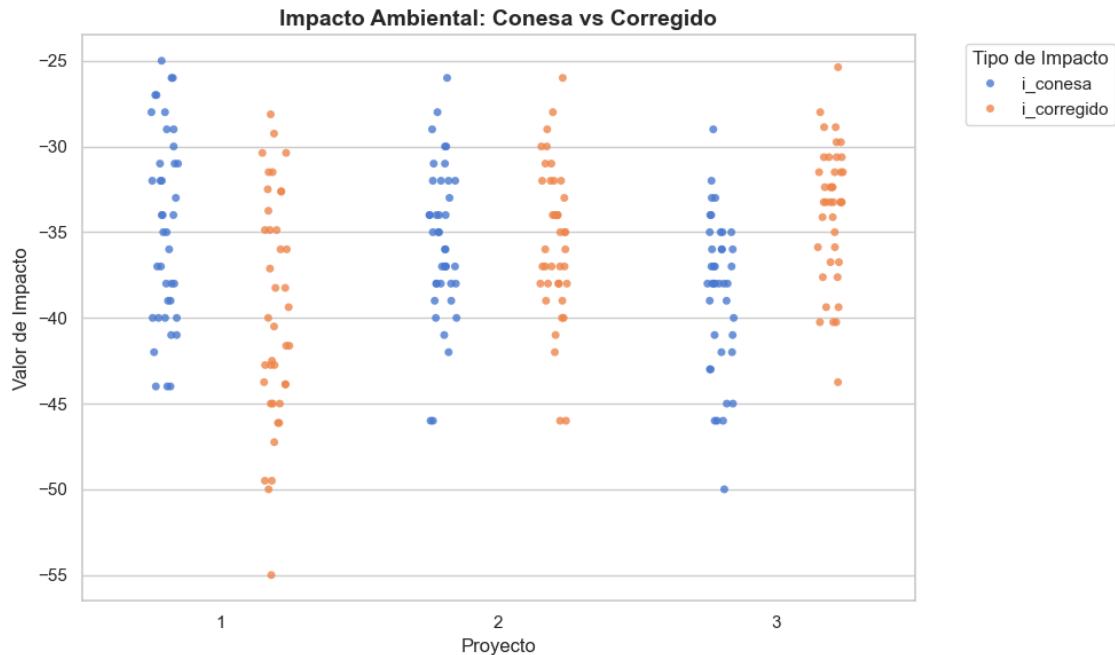
# Supongamos que tu DataFrame 'impactos' ya tiene:
# columnas: 'id_proyecto', 'i_conesa', 'i_corregido'

# Configuración de estilo
sns.set(style="whitegrid", palette="muted")

# Crear figura
plt.figure(figsize=(10,6))

# Gráfico comparativo: i_conesa vs i_corregido
sns.stripplot(
    data=impactos.melt(
        id_vars="id_proyecto",
        value_vars=["i_conesa", "i_corregido"],
        var_name="Tipo",
        value_name="Impacto"
    ),
    x="id_proyecto",
    y="Impacto",
    hue="Tipo",          # Diferencia por tipo de impacto
    dodge=True,
    jitter=True,
    alpha=0.8
)

# Personalización
plt.title("Impacto Ambiental: Conesa vs Corregido", fontsize=14,
        fontweight="bold")
plt.xlabel("Proyecto")
plt.ylabel("Valor de Impacto")
plt.legend(title="Tipo de Impacto", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



1.2.2 Gráfica de Resiliencia Normalizada por Proyecto

Este bloque genera una visualización de los valores de **Resiliencia Normalizada (re_norm)** agrupados por proyecto. El objetivo es comparar la capacidad de recuperación de cada ecosistema.

- **Eje X:** Identificador del proyecto (`id_proyecto`).
- **Eje Y:** Resiliencia normalizada (`re_norm`), valores entre 0 y 1.
- **Colores:** Diferencian cada proyecto.
- **Interpretación:**
 - Proyecto 1 (Warnes): Alta resiliencia (0.75–1.0).
 - Proyecto 2 (Qollpana): Resiliencia moderada (~0.5).
 - Proyecto 3 (Altiplano): Baja resiliencia (~0.25–0.3).

La gráfica muestra cómo la resiliencia varía según el contexto territorial, confirmando que ecosistemas biodiversos absorben mejor los impactos, mientras que ecosistemas frágiles los amplifican.

```
[16]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

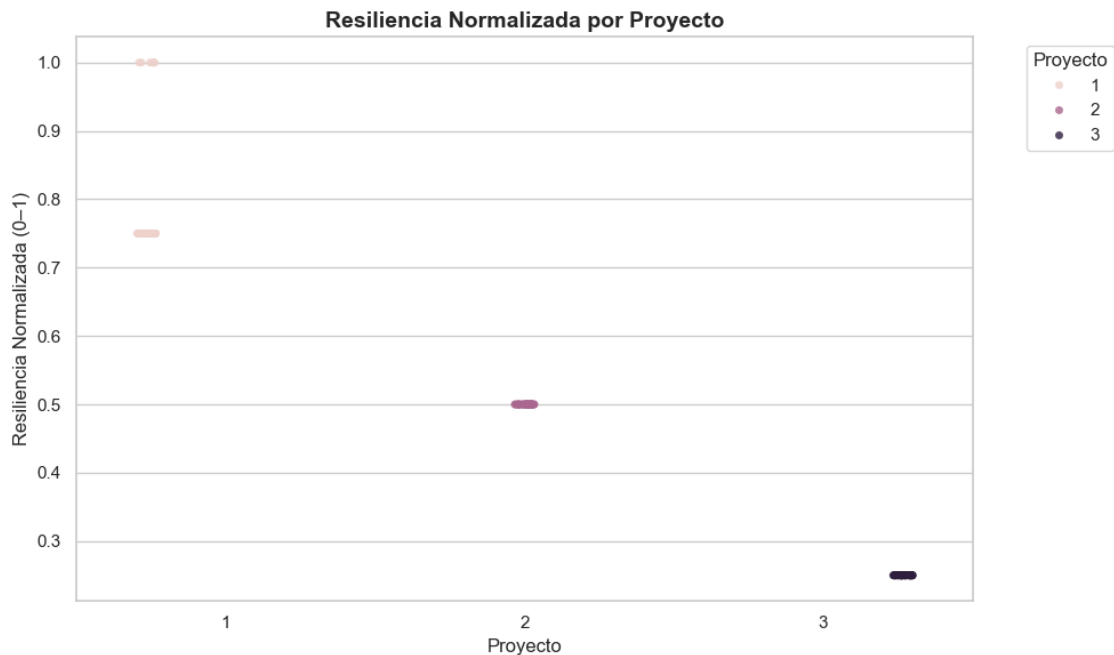
# Cargar datos
impactos = pd.read_csv("fact_impactos_final.csv")

# Configuración de estilo
sns.set(style="whitegrid", palette="muted")

# Gráfica de puntos: resiliencia normalizada por proyecto
```

```
plt.figure(figsize=(10,6))
sns.stripplot(
    data=impactos,
    x="id_proyecto",
    y="re_norm",          # valor de resiliencia normalizada
    hue="id_proyecto",   # colores por proyecto
    dodge=True,
    jitter=True,
    alpha=0.8
)

# Personalización
plt.title("Resiliencia Normalizada por Proyecto", fontsize=14,
        fontweight="bold")
plt.xlabel("Proyecto")
plt.ylabel("Resiliencia Normalizada (0-1)")
plt.legend(title="Proyecto", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



1.2.3 Cálculo del Factor de Resiliencia (FR)

Este bloque calcula el **Factor de Resiliencia (FR)** para cada proyecto, ajustando el impacto según la resiliencia normalizada.

- **Fórmula:**

$$[FR = 1 + (0.5 - RE_{\{norm\}})]$$

donde: - ($RE_{\{norm\}}$): resiliencia normalizada. - (): sensibilidad (ejemplo: 0.5).

- **Resultados promedio por proyecto:**
 - Proyecto 1 (Warnes): $FR = 0.856 \rightarrow$ Impactos **reducidos**.
 - Proyecto 2 (Qollpana): $FR = 1.000 \rightarrow$ Impactos **estables**.
 - Proyecto 3 (Altiplano): $FR = 1.125 \rightarrow$ Impactos **amplificados**.
- **Interpretación:**
 - Alta resiliencia reduce impactos.
 - Resiliencia intermedia mantiene impactos.
 - Baja resiliencia amplifica impactos.

Este cálculo permite ajustar el **Impacto Conesa corregido ($i_{\text{corregido}}$)** y obtener una visión más realista del riesgo ambiental.

```
[17]: import pandas as pd

# Cargar datos
impactos = pd.read_csv("fact_impactos_final.csv")

# Definir beta
beta = 0.5

# Calcular factor de resiliencia
impactos['FR'] = 1 + beta * (0.5 - impactos['re_norm'])

# Agrupar por proyecto y calcular promedio de FR
fr_proyecto = impactos.groupby('id_proyecto')['FR'].mean().reset_index()

print(fr_proyecto)
```

	id_proyecto	FR
0	1	0.85625
1	2	1.00000
2	3	1.12500

[]:

1.2.4 Gráfica de Impacto Conesa por Proyecto

Este bloque genera una visualización de los valores de **Impacto Conesa (i_{conesa})** agrupados por proyecto. El objetivo es comparar la distribución de impactos negativos entre los distintos proyectos.

- **Eje X:** Identificador del proyecto (id_{proyecto}).
- **Eje Y:** Valor de impacto Conesa (i_{conesa}).
- **Colores:** Diferencian cada proyecto.
- **Interpretación:**
 - Proyecto 1 (Warnes): Impactos moderados.
 - Proyecto 2 (Qollpana): Impactos moderados-altos.

- Proyecto 3 (Altiplano): Impactos más severos.

La gráfica permite identificar qué proyectos generan impactos más críticos y compararlos en un mismo plano.

1.2.5 Gráfica de Resiliencia Normalizada por Proyecto

Este bloque genera una visualización de los valores de **Resiliencia Normalizada (re_norm)** agrupados por proyecto. El objetivo es comparar la capacidad de recuperación de cada ecosistema.

- **Eje X:** Identificador del proyecto (**id_proyecto**).
- **Eje Y:** Resiliencia normalizada (**re_norm**), valores entre 0 y 1.
- **Colores:** Diferencian cada proyecto.
- **Interpretación:**
 - Proyecto 1 (Warnes): Alta resiliencia (0.75–1.0).
 - Proyecto 2 (Qollpana): Resiliencia moderada (~0.5).
 - Proyecto 3 (Altiplano): Baja resiliencia (~0.25–0.3).

La gráfica muestra cómo la resiliencia varía según el contexto territorial, confirmando que ecosistemas biodiversos absorben mejor los impactos, mientras que ecosistemas frágiles los amplifican.

1.2.6 Cálculo del Factor de Resiliencia (FR)

Este bloque calcula el **Factor de Resiliencia (FR)** para cada proyecto, ajustando el impacto según la resiliencia normalizada.

- **Fórmula:**

$$[FR = 1 + (0.5 - RE_{\{norm\}})]$$

donde: - ($RE_{\{norm\}}$): resiliencia normalizada. - (): sensibilidad (ejemplo: 0.5).

- **Resultados promedio por proyecto:**
 - Proyecto 1 (Warnes): $FR = 0.856 \rightarrow$ Impactos **reducidos**.
 - Proyecto 2 (Qollpana): $FR = 1.000 \rightarrow$ Impactos **estables**.
 - Proyecto 3 (Altiplano): $FR = 1.125 \rightarrow$ Impactos **amplificados**.

Este cálculo permite ajustar el **Impacto Conesa corregido (i_corregido)** y obtener una visión más realista del riesgo ambiental.

1.2.7 Impacto Corregido (I_corregido)

El **Impacto Corregido (i_corregido)** es el valor central de la metodología MEIA-R, porque integra el impacto base con el factor de resiliencia.

- **Fórmula:**

$$[i_{\{corregido\}} = i_{\{conesa\}} \times FR]$$

- **Interpretación:**
 - Si $FR < 1 \rightarrow$ el impacto se **reduce** (ecosistema robusto).
 - Si $FR = 1 \rightarrow$ el impacto se **mantiene** (resiliencia intermedia).
 - Si $FR > 1 \rightarrow$ el impacto se **amplifica** (ecosistema frágil).
- **Ejemplo aplicado:**

- Warnes: impactos moderados, resiliencia alta → *i_corregido* menor que *i_conesa*.
- Qollpana: resiliencia intermedia → *i_corregido* *i_conesa*.
- Altiplano: resiliencia baja → *i_corregido* más severo que *i_conesa*.

Este valor es el **KPI principal** para reportes ejecutivos, porque refleja la realidad territorial y permite priorizar acciones de mitigación.

```
[30]: import pandas as pd

# Cargar datos
impactos = pd.read_csv("fact_impactos.csv")

# Asegurar que todos los signos sean negativos
impactos['signo'] = -1

# Recalcular i_conesa con las 10 variables de Conesa
variables_conesa = ['i','ex','mo','pe','rv','si','ac','ef','pr','mc']
impactos['i_conesa'] = impactos[variables_conesa].sum(axis=1) *
↳ impactos['signo']

# Calcular Impacto Corregido usando el factor de resiliencia
impactos['i_corregido'] = impactos['i_conesa'] * impactos['f_resiliencia']

# Guardar archivo actualizado
impactos.to_csv("fact_impactos.csv", index=False)

# --- Para fact_impactos_final.csv ---
impactos_final = pd.read_csv("fact_impactos_final.csv")

# Asegurar que todos los signos sean negativos
impactos_final['signo'] = -1

# Recalcular i_conesa
impactos_final['i_conesa'] = impactos_final[variables_conesa].sum(axis=1) *
↳ impactos_final['signo']

# Calcular Impacto Corregido
impactos_final['i_corregido'] = impactos_final['i_conesa'] *
↳ impactos_final['f_resiliencia']

# Guardar archivo actualizado
impactos_final.to_csv("fact_impactos_final.csv", index=False)

print("Archivos recalculados con i_conesa e i_corregido actualizados.")
```

Archivos recalculados con *i_conesa* e *i_corregido* actualizados.

```
[ ]:
```